

Fine-tuning FunctionGemma for Qdrant Tool Calling - Cell Explanations

Cell 1: Package Installation

Code:

```
!pip install torch transformers datasets accelerate evaluate trl protobuf sentencepiece huggingface-hub
```

Description:

This cell installs all the essential Python packages required for fine-tuning the language model. The torch package provides the deep learning framework, transformers enables loading and working with pre-trained models from HuggingFace, datasets facilitates data handling and preprocessing, accelerate optimizes training on various hardware configurations, evaluate offers metrics for model assessment, trl provides the supervised fine-tuning trainer, while protobuf and sentencepiece handle tokenization for the Gemma model family, and huggingface-hub manages model versioning and downloading.

Cell 2: HuggingFace Authentication (Commented Out)

Code:

```
# from huggingface_hub import login  
# login()
```

Description:

This commented-out cell would authenticate the user with HuggingFace Hub using their access token. Authentication is necessary when accessing gated models or pushing fine-tuned models to the HuggingFace repository. The code is disabled here, suggesting either the model is publicly accessible or authentication was handled externally.

Cell 3: Model Configuration

Code:

```
base_model = "google/functiongemma-270m-it"  
learning_rate = 5e-5
```

Description:

This cell establishes the fundamental training parameters by setting the base model identifier to Google's FunctionGemma 270M instruction-tuned variant, which is specifically designed for function calling tasks, and configures the learning rate to 5e-5, a relatively conservative value that ensures stable training without overshooting optimal weight updates during gradient descent.

Cell 4: Training Data Creation

Code:

```
qdrant_tool_calling = [  
    {"user_content": "Create a new collection called documents with 384 dimensional  
    vectors using cosine distance", "tool_name": "create_collection", "tool_arguments"  
    : "{\\"collection_name\\": \\"documents\\", \\"vector_size\\": 384, \\"distance\\": \\"Cos  
    ine\\"}"},  
    {"user_content": "Add a point with id 1 to the images collection with vector [0  
.5, 0.3] and metadata category cat", "tool_name": "upsert_points", "tool_arguments"  
    : "{\\"collection_name\\": \\"images\\", \\"points\\": [{\\"id\\": 1, \\"vector\\": [0.5, 0  
.3], \\"payload\\": {\\"category\\": \\"cat\\"}}]}"},  
    # ... more examples  
]
```

Description:

This cell constructs a comprehensive training dataset containing user queries paired with their corresponding Qdrant API function calls and properly formatted JSON arguments. Each dictionary entry represents a complete training example with `user_content` describing the natural language request, `tool_name` specifying which Qdrant operation to invoke, and `tool_arguments` containing the exact JSON parameters needed to execute that operation. The dataset covers various Qdrant operations including collection creation, point insertion, retrieval, searching, scrolling, and deletion, providing diverse examples that teach the model to map natural language instructions to correct function calls with appropriate arguments.

Cell 5: Tool Definitions and Dataset Preparation

Code:

```
import json
from datasets import Dataset
from transformers.utils import get_json_schema

def create_collection(collection_name: str, vector_size: int, distance: str) ->
str:
    """Creates a new collection in Qdrant with specified vector configuration."""
    # ... function implementation

# ... more tool definitions

TOOLS = [get_json_schema(fn) for fn in [create_collection, upsert_points, get_point,
search_points, scroll_points, delete_points]]

# Format training data
formatted_data = []
for item in qdrant_tool_calling:
    messages = [
        {"role": "user", "content": item["user_content"]},
        {"role": "assistant", "tool_calls": [{"id": "1", "type": "function", "function": {"name": item["tool_name"], "arguments": item["tool_arguments"]}]}}]
    formatted_data.append({"messages": messages})

dataset = Dataset.from_list(formatted_data).train_test_split(test_size=0.2)
```

Description:

This cell establishes the complete tool infrastructure by defining Python functions for each Qdrant operation with detailed docstrings and type annotations, which are then converted into JSON schema format that the model can understand and use during training. The `get_json_schema` utility extracts function signatures, parameter types, and descriptions to create standardized tool definitions. The training data is transformed from the raw dictionary format into a conversational message structure following the chat template convention where each example contains a user message with the natural language request and an assistant response containing the structured tool call with function name and arguments. The formatted dataset is then split with 80% allocated for training and 20% reserved for evaluation, ensuring the model's performance can be assessed on unseen examples during fine-tuning.

Cell 6: Model Loading and Validation

Code:

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    base_model,
    dtype="auto",
    device_map="auto",
    attn_implementation="eager"
)
tokenizer = AutoTokenizer.from_pretrained(base_model)

print(f"Device: {model.device}")
print(f"DType: {model.dtype}")
print("--- dataset input ---")
print(json.dumps(dataset["train"][0], indent=2))
debug_msg = tokenizer.apply_chat_template(dataset["train"][0]["messages"], tools
=TOOLS, add_generation_prompt=False, tokenize=False)
print(debug_msg)
```

Description:

This cell loads the pre-trained FunctionGemma model using automatic data type selection and device mapping for optimal memory usage across available hardware, while the eager attention implementation ensures compatibility though it may be slower than optimized alternatives. The tokenizer is loaded separately to handle text encoding and decoding using the model's specific vocabulary and special tokens. The validation section prints device placement and data type information to verify correct hardware utilization, displays the first training example to inspect the data structure, and applies the chat template with tool definitions to show exactly how the model will see the formatted input during training, confirming that the message structure, tool schemas, and special tokens are correctly assembled before beginning the fine-tuning process.

Cell 7: Pre-training Evaluation Function

Code:

```
def check_success_rate():
    success_count = 0
    for idx, item in enumerate(dataset['test']):
        messages = [item["messages"][0], item["messages"][1]]
        inputs = tokenizer.apply_chat_template(messages, tools=TOOLS, add_generation
        _prompt=True, return_dict=True, return_tensors="pt")
        out = model.generate(**inputs.to(model.device), pad_token_id=tokenizer.eos_t
        oken_id, max_new_tokens=128)
        output = tokenizer.decode(out[0][len(inputs["input_ids"][0]):], skip_special
        _tokens=True)

        try:
            parsed = json.loads(output)
            gt_tool = item["messages"][1]["tool_calls"][0]["function"]
            pred_tool = parsed[0]

            if pred_tool["name"] == gt_tool["name"] and json.loads(pred_tool["argume
            nts"]) == json.loads(gt_tool["arguments"]):
                success_count += 1
        except:
            pass

    print(f"Success rate: {success_count}/{len(dataset['test'])} = {success_count/
    len(dataset['test'])*100:.2f}%")
```

Description:

This function evaluates the model's ability to generate correct tool calls by iterating through the test dataset and comparing predicted outputs against ground truth labels. For each test example, it constructs the input prompt with user message and tool definitions, generates the model's response with a maximum of 128 new tokens, extracts only the newly generated content by slicing away the input tokens, and attempts to parse the output as JSON to extract the predicted function name and arguments. The evaluation considers a prediction successful only when both the tool name matches exactly and the parsed JSON arguments are structurally identical to the expected values, accounting for potential ordering differences. The function handles parsing failures gracefully by catching exceptions and counting them as incorrect predictions, finally reporting the overall success rate as both a fraction and percentage to provide a clear baseline metric before fine-tuning begins.

Cell 8: Training Configuration

Code:

```
from trl import SFTConfig

torch_dtype = model.dtype

args = SFTConfig(
    output_dir="./functiongemma-270m-it-qdrant-ft",
    max_length=512,
    packing=False,
    num_train_epochs=8,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    gradient_checkpointing=True,
    optim="adamw_torch_fused",
    learning_rate=learning_rate,
    lr_scheduler_type="cosine",
    warmup_ratio=0.1,
    bf16=True,
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    push_to_hub=False,
    report_to="none",
)
```

Description:

This cell configures all hyperparameters and training settings using the `SFTConfig` class, which specializes in supervised fine-tuning workflows. The configuration sets the output directory for checkpoints and logs, limits sequence length to 512 tokens to balance memory usage and context coverage, disables packing to keep examples separate for clearer learning, and schedules training for 8 complete passes through the dataset. The batch size of 4 per device combined with 4 gradient accumulation steps creates an effective batch size of 16, allowing stable training while managing memory constraints through gradient checkpointing which trades computation for reduced memory by recomputing activations during backpropagation. The optimizer uses the fused AdamW implementation for faster training, applies the previously defined learning rate with a cosine decay schedule that gradually reduces the rate over time, and includes a warmup phase covering 10% of training to prevent early instability. The configuration enables bfloat16 precision for memory efficiency and faster computation, logs metrics every 10 steps for monitoring, performs evaluation and saves checkpoints at the end of each epoch, and disables HuggingFace Hub uploading and external reporting to keep training local.

Cell 9: Model Training

Code:

```
from trl import SFTTrainer

try:
    trainer = SFTTrainer(
        model=model,
        args=args,
        train_dataset=dataset['train'],
        eval_dataset=dataset['test'],
        processing_class=tokenizer,
    )

    print("Trainer created successfully")
    trainer.train()
    print("Training completed")

except Exception as e:
    print(f"Error occurred: {type(e).__name__}")
    print(f"Error message: {str(e)}")
    import traceback
    traceback.print_exc()
```

Description:

This cell instantiates the SFTTrainer object with the loaded model, configuration settings, training and evaluation datasets, and tokenizer for text processing, then executes the complete fine-tuning process. The try-except block provides robust error handling that catches any exceptions during trainer initialization or training execution, printing both the error type and detailed message along with the full traceback to facilitate debugging if issues arise. The trainer automatically handles the training loop including forward passes, loss calculation, backpropagation, optimizer steps, learning rate scheduling, gradient accumulation, checkpointing, and periodic evaluation on the test set according to the specified configuration. Status messages confirm successful trainer creation and training completion, allowing easy monitoring of the fine-tuning pipeline's progression through its major phases.

Cell 10: Training Visualization

Code:

```
import matplotlib.pyplot as plt

train_losses = [log['loss'] for log in trainer.state.log_history if 'loss' in log]
eval_losses = [log['eval_loss'] for log in trainer.state.log_history if 'eval_loss' in log]

epoch_train = [i/len(dataset['train']) for i in range(len(train_losses))]
epoch_eval = [i+1 for i in range(len(eval_losses))]

plt.plot(epoch_train, train_losses, label='Training Loss')
plt.plot(epoch_eval, eval_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True)
plt.show()
```

Description:

This cell creates a visualization of the training progression by extracting loss values from the trainer's logged history and plotting them against training epochs. The training losses are recorded at regular intervals throughout each epoch based on the `logging_steps` configuration, while evaluation losses are computed once at the end of each epoch. The epoch values for training losses are calculated by dividing the step count by the dataset size to properly align them on the time axis, while evaluation losses are simply incremented by whole epoch numbers. The resulting line plot displays both training and validation curves together, allowing visual assessment of model convergence, identification of potential overfitting where validation loss increases while training loss decreases, and verification that the learning rate schedule and other hyperparameters are producing the expected training dynamics with a grid overlay and legend for improved readability.

Cell 11: Post-training Evaluation

Code:

```
check_success_rate()
```

Description:

This cell re-runs the evaluation function defined earlier to measure the fine-tuned model's performance on the test dataset, providing a direct comparison with the pre-training baseline metrics. By executing the same success rate calculation after training completes, it quantifies the improvement gained from fine-tuning in terms of exact match accuracy for both function names and arguments. This final evaluation demonstrates whether the model has successfully learned to map natural language Qdrant queries to correct tool calls and properly formatted JSON parameters, validating the effectiveness of the fine-tuning process and the quality of the training data.

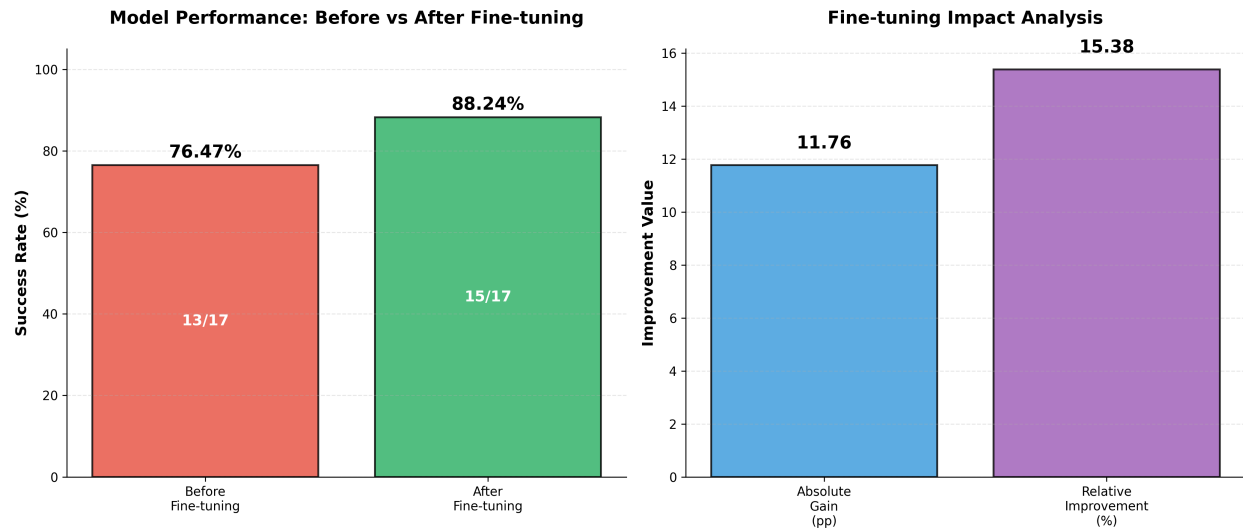
Fine-tuning Results and Performance Analysis

Performance Summary

Metric	Before Fine-tuning	After Fine-tuning
Correct Predictions	13/17	15/17
Success Rate	76.47%	88.24%
Improvement Metrics	Value	
Absolute Gain	11.76 pp	
Relative Improvement	15.38%	
Additional Correct	+2 predictions	

The fine-tuning process demonstrated measurable improvement in the model's ability to correctly generate Qdrant tool calls. Starting from a baseline of 76.47% accuracy with 13 out of 17 correct predictions, the model achieved 88.24% accuracy after fine-tuning, correctly handling 15 out of 17 test cases. This represents an absolute gain of 11.76 percentage points and a relative improvement of 15.38%. The two additional correct predictions indicate that the model successfully learned to better map natural language instructions to the appropriate Qdrant API functions with correctly formatted JSON arguments.

Visual Performance Comparison



The visualization clearly demonstrates the positive impact of fine-tuning on model performance. The left panel shows the direct comparison between pre and post fine-tuning success rates, while the right panel quantifies the improvement achieved. This fine-tuning approach successfully enhanced the model's capability to understand and execute Qdrant vector database operations through natural language instructions.